

Contextual Bandit – LinUCB

Kod ve Algoritma Üzerine Teknik Analiz Raporu

Kapsam: analiz.ipynb, contextual_bandit_dataset.csv

Hazırlayan: Betül Beyza Demirel

1. Çalışmanın Amacı ve Veri Yapısı

1.1 Contextual Bandit nedir?

Contextual bandit, bir algoritmanın elindeki bilgiye bakarak birkaç seçenek arasından birini seçtiği ve seçimin sonucunu gözlemleyerek zamanla daha iyi seçimler yapmayı öğrendiği bir karar verme yapısıdır. Klasik makine öğrenmesinden farkı şudur: model önceden hazır bir eğitim setiyle eğitilmez, her kullanıcıda 'tahmin et → seç → sonucu gör → güncelle' döngüsü canlı olarak işler. Bu yüzden bandit algoritmaları 'online öğrenme' ailesindedir.

1.2 Bu projede context, arm ve reward neyi temsil ediyor?

Kavram	Bu projedeki karşılığı
Context (x)	Her kullanıcının özellik vektörü: yaş, oturum sayısı, ortalama harcama ve one-hot kodlanmış cinsiyet/cihaz/bölge/saat bilgisi (toplam 15 sütun).
Arm	Kullanıcıya gösterilecek 5 farklı reklam/aksiyon seçeneği (arm_0 ... arm_4).
Reward	Kullanıcının seçilen arm'a verdiği tepki: 0 veya 1.

Veri setinde her satır bir kullanıcıyı temsil eder ve reward_arm_0 ... reward_arm_4 sütunlarının hepsi önceden bellidir. Gerçek hayatta yalnızca seçilen arm'ın reward'ı görülür; burada ise simülasyon yapabilmek için tüm arm'ların reward'ı veri setine önceden yerleştirilmiştir. Kod yalnızca seçtiği arm'ın reward'ını kullanır (rewards[i, chosen_arm]), diğerlerine hiç bakmaz — bu da gerçek bandit senaryosunu doğru şekilde simüle eder.

1.3 Feature ve reward sütunları nasıl kullanılıyor?

user_id ve reward sütunları context'e dahil edilmez (feature_cols oluşturulurken bilinçli olarak çıkarılır), geri kalan 15 sütun context matrisi X 'i oluşturur. Bu 15 sütun: age, session_count, avg_purchase (sayısal) ile gender_M, device_mobile, device_tablet, region_* (6 sütun), time_of_day_* (3 sütun) (kategorik, one-hot) sütunlarıdır. reward_arm_0 ... reward_arm_4 sütunları ayrı bir rewards matrisinde tutulur ve yalnızca seçim yapıldıktan sonra 'doğru cevabı kontrol etmek' için kullanılır.

1.4 Preprocessing: One-Hot Encoding ve StandardScaler neden kullanıldı?

One-Hot Encoding (pd.get_dummies):

gender, device, region, time_of_day gibi kategorik sütunlar metin (F/M, mobile/tablet/desktop gibi) içerir; ancak LinUCB'nin matris işlemleri ($A\theta=b$, $x^T A^{-1}x$) yalnızca sayılarla çalışabilir. One-hot encoding her kategoriye 0/1'lerden oluşan ayrı bir sütuna çevirir. drop_first=True ile her kategorik değişkenin ilk sınıfı düşürülür (örn. gender_F sütunu yok, sadece gender_M var), çünkü diğer tüm sütunlar 0 ise o satırın otomatik olarak düşürülen sınıfa ait olduğu anlaşılır. Bu, gereksiz bir sütunu (ve modelde çoklu doğrusal bağlantı riskini) ortadan kaldırır.

StandardScaler:

age, session_count, avg_purchase sütunları çok farklı ölçeklerde (örneğin yaş 18-70 arası). One-hot sütunları zaten 0/1 aralığındadır ama sayısal sütunlar ölçeklenmezse, büyük değerli sütun (avg_purchase gibi) $x^T A^{-1}x$ ve $\text{np.dot}(x, \theta)$ işlemlerinde diğer sütunlara göre orantısız şekilde baskın çıkar. StandardScaler her sütunu ortalaması 0, standart sapması 1 olacak şekilde yeniden ölçekler, böylece hiçbir feature diğerine göre ağırlık avantajı kazanmaz.

2. LinUCB'nin Temel Mantığı

LinUCB, her arm için ayrı bir lineer ilişki öğrenir: reward'ın context'e göre yaklaşık olarak doğrusal değiştiğini varsayar. Algoritmanın kalbi şu tek cümledir: 'Beklenen kazanç ne kadar yüksekse VE bu tahminden ne kadar az eminsem, o arm'ı denemeye o kadar değer.'. Akış, her kullanıcı için aşağıdaki adımları tekrar eder:

- Yeni bir kullanıcı/context gelir (x).
- Her arm için ayrı ayrı: o ana kadar öğrenilenlere göre bir tahmin (prediction) hesaplanır.
- Yine her arm için: bu tahminin ne kadar güvenilir olduğu, yani belirsizlik (uncertainty) hesaplanır.
- prediction + uncertainty toplanarak o arm'ın score'u oluşturulur.
- En yüksek score'a sahip arm seçilir (argmax).
- Seçilen arm kullanıcıya gösterilir ve gerçek reward gözlemlenir (0 veya 1).
- Sadece seçilen arm'ın A ve b değerleri, bu yeni gözlemle güncellenir.
- Bir sonraki kullanıcı geldiğinde, o arm için artık güncellenmiş A ve b kullanılır.

Neden hem prediction hem uncertainty?

Sadece prediction'a bakılsaydı algoritma çok hızlı 'en iyi bildiği' arm'da saplanır kalırdı (exploitation) ve az denenmiş ama aslında iyi olabilecek arm'ları hiç keşfetmezdi.

uncertainty terimi, az denenmiş arm'lara ekstra bir 'bonus puan' vererek onların da zaman zaman seçilmesini sağlar (exploration).

Böylece LinUCB, 'bildiğini kullanma' ile 'bilmediğini keşfetme' arasında matematiksel bir denge kurar.

3. Kod ile Algoritmanın Eşleştirilmesi

Kodda geçen ana değişkenlerin algoritmadaki ve np dizisi olarak koddaki karşılıkları:

Değişken	Algoritmadaki anlamı	Koddaki rolü
X	Tüm kullanıcıların context matrisi	Shape (8000, 15) — satır: kullanıcı, sütun: feature
x	Tek bir kullanıcının context vektörü	$X[i]$ — shape (15,) o an işlenen kullanıcı
A	Her arm için biriken 'bilgi/güven' matrisi (xx^T toplamı)	5 elemanlı liste, her biri (15,15) matris, başlangıçta birim matris (np.eye)
b	Her arm için biriken 'ödül ağırlıklı context' vektörü	5 elemanlı liste, her biri (15,) vektör, başlangıçta sıfır
theta	O arm için öğrenilen katsayılar (ağırlıklar)	np.linalg.solve(A[arm], b[arm]) ile hesaplanır
prediction	theta ile x'in doğrusal kombinasyonu → beklenen reward	np.dot(x, theta)
uncertainty	Tahminin ne kadar az bilindiği (güven aralığı genişliği)	$\sqrt{x^T A^{-1} x}$
alpha	Belirsizliğe verilen ağırlık (exploration katsayısı)	Sabit değer, alpha = 1
score	Arm'ı seçmek için kullanılan nihai puan	prediction + alpha * uncertainty
reward	Seçilen arm'ın gerçek sonucu	rewards[i, chosen_arm] / 0 ya da 1

Kısaca: X ve x çoğul/tekil ilişkisindedir (X tüm kullanıcılar, x o anki kullanıcı). A ve b, arm'ların 'hafızasıdır' — her arm kendi A ve b'sini biriktirir. theta, bu hafızadan çıkarılan katsayı setidir. prediction ve uncertainty, theta ve A üzerinden hesaplanıp score'da birleşir; score'un en büyük olduğu arm seçilir ve döngü reward ile A/b'yi güncelleyerek kapanır.

4. LinUCB'nin Matematiksel Olarak Önemli Kısım

```
theta = np.linalg.solve(A[arm], b[arm])
prediction = np.dot(x, theta)
A_inv = np.linalg.inv(A[arm])
uncertainty_value = np.dot(x, np.dot(A_inv, x))
uncertainty = np.sqrt(uncertainty_value)
score = prediction + alpha * uncertainty
```

4.1 np.linalg.solve() ne yapıyor? $A\theta = b$ ilişkisi nedir?

LinUCB, her arm için reward'ın context'e doğrusal bağlı olduğunu varsayar: $\text{reward} \approx x \cdot \theta$. Elimizdeki A ve b, o ana kadar gözlemlenen tüm (x, reward) çiftlerinin özetidir. En iyi θ , $A\theta = b$ denklemini sağlayan θ 'dır. Bu, doğrusal regresyonun kapalı-form çözümüne karşılık gelir. np.linalg.solve(A, b) bu denklem sistemini çözüp θ 'yı doğrudan verir.

Burada dikkat edilmesi gereken nokta: solve() işlemi, matematiksel olarak $\theta = A^{-1}b$ 'ye eşdeğerdir, ama A'nın tersini açıkça hesaplamadan, sayısal olarak daha kararlı bir şekilde aynı sonucu bulur. Yani `theta = np.linalg.solve(A[arm], b[arm])` satırı, 'A'nın tersini alıp b ile çarpma' işleminin daha güvenilir bir versiyonudur.

4.2 np.linalg.inv() neden kullanılıyor? $x^T A^{-1} x$ neyi ölçüyor?

prediction için A'nın tersine ihtiyaç yoktu (solve yeterliydi), ama uncertainty için gerçekten A^{-1} 'in kendisi lazım, çünkü hesaplanan şey $x^T A^{-1} x$ — yani x vektörünün A^{-1} matrisi üzerinden 'kendisiyle' yaptığı bir iç çarpım (kuadratik form). Bu ifade, x yönünde A matrisinin ne kadar zayıf/az bilgi biriktirmiş olduğunu ölçer. A içinde x'e benzer yönlerde çok veri birikmişse yani bu tür kullanıcılar bu arm için daha önce çok görülmüşse A büyür, A^{-1} küçülür, dolayısıyla $x^T A^{-1} x$ küçük çıkar, belirsizlik azalır. Tam tersi, bu context'e benzer az örnek görülmüşse A hâlâ birim matrise yakındır, A^{-1} büyük kalır, $x^T A^{-1} x$ büyük çıkar, belirsizlik yüksektir.

uncertainty_value negatif olamaz A simetrik ve pozitif tanımlıdır, bu yüzden karekökü (`np.sqrt`) alınarak standart sapmaya benzer bir belirsizlik büyüklüğü elde edilir; bu istatistikte güven aralığı genişliğinin kareköküyle orantılı olması fikrine dayanır.

4.3 prediction ve uncertainty neden toplanıyor? Exploration/exploitation bağlantısı

`score = prediction + alpha * uncertainty` ifadesi, istatistikteki 'üst güven sınırı' (upper confidence bound) fikrinin doğrudan uygulamasıdır: sadece en iyi tahmin edilen değere değil, bu tahmin ne kadar hatalı olabilir, en iyi ihtimalle ne kadar yüksek olabilir sorusuna göre karar verilir. prediction yüksekse arm zaten iyi görünüyor demektir (exploitation), uncertainty yüksekse arm hakkında yeterince veri yok demektir ve bu arm'a bir şans daha verilir (exploration). alpha bu iki eğilim arasındaki dengeyi ayarlayan bir kadrandır. alpha büyütülürse algoritma daha fazla keşif yapar, küçültülürse öğrendiğine daha çok güvenip aynı arm'larda kalma eğilimi artar. Bu projede alpha = 1 sabit tutulmuştur.

4.4 Küçük sayısal örnek: $x^T A^{-1} x$

2 feature'lı, sadeleştirilmiş bir örnek üzerinden gösterelim. Bir arm için A ve gelen context x şu şekilde olsun:

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$$

$$x = [1, 2]$$

Adım adım:

- $A^{-1} = \begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix}$ (A köşegen olduğu için tersi de köşegendir, elemanların tersi alınır)
- $A^{-1} \cdot x = [0.5 \times 1 + 0 \times 2, 0 \times 1 + 0.5 \times 2] = [0.5, 1]$
- $x^T \cdot (A^{-1} x) = (1 \times 0.5) + (2 \times 1) = 0.5 + 2 = 2.5$
- $\text{uncertainty} = \sqrt{2.5} \approx 1.58$

A henüz birim matrise yakınsa (yani bu arm çok az denendiyse) A^{-1} büyük kalır ve uncertainty yüksek çıkar. Arm çok kez seçilip A büyüdükçe (aşağıdaki bölümde göreceğimiz gibi A'ya sürekli xx^T eklenir), A^{-1} küçülür ve aynı x için uncertainty giderek düşer — yani algoritma o arm hakkında 'daha emin' hâle gelir.

5. Algoritmanın Öğrenme / Güncelleme Kısmı

```
A[chosen_arm] = A[chosen_arm] + np.outer(x, x)
b[chosen_arm] = b[chosen_arm] + reward * x
```

Öğrenme, LinUCB'de tam olarak bu iki satırda gerçekleşir. Bu adıma kadarki her şey (theta, prediction, uncertainty, score) sadece o anki A ve b'ye bakarak bir karar üretmekle ilgiliydi. Asıl öğrenme, kararın sonucu (reward) geldikten sonra bu iki satırla A ve b'nin güncellenmesidir.

5.1 np.outer(x, x) nedir?

$\text{np.outer}(x, x)$, x vektörünü kendisiyle çarparak bir matris üretir. Sonucun (i, j) hücresi $x[i] \times x[j]$ 'dir. Bu matris, x'in tüm feature çiftleri arasındaki ilişkiyi ve her feature'ın kendi karesini köşegende tutar. A matrisine bu matrisi eklemek, bu context'i gördük bilgisini A'nın hafızasına işlemek anlamına gelir.

5.2 A neden güncelleniyor, b neden güncelleniyor? Aralarındaki fark ne?

A, yalnızca hangi context'lerin (x'lerin) o arm için görüldüğünü biriktirir reward'dan bağımsızdır, sadece bu tür kullanıcı bu arm'a kaç kez ve nasıl geldi bilgisini tutar. Bu yüzden A'nın büyüklüğü doğrudan güveni yani düşük uncertainty'yi besler. b ise reward ile ağırlıklandırılmış context'leri biriktirir: $\text{reward} * x$. $\text{reward} = 1$ olduğunda x aynen b'ye eklenir ('bu context iyi sonuç verdi, hatırla'); $\text{reward} = 0$ olduğunda $\text{reward} * x = 0$ olduğu için b hiç değişmez ('bu context'ten olumlu bir sinyal gelmedi'). Kısacası A 'ne kadar biliyoruz', b ise 'ne öğrendik/hangi yönde öğrendik' sorusuna karşılık gelir; $\theta = A^{-1}b$ ikisini birleştirerek nihai tahmini üretir.

5.3 Küçük örnek: A ve b nasıl değişir?

Bir arm için başlangıç: $A = \text{birim matris} [[1,0],[0,1]]$, $b = [0,0]$. Bir kullanıcı geldi, context $x = [1, 2]$, bu arm seçildi ve $\text{reward} = 1$ gözlemlendi:

- $\text{np.outer}(x, x) = [[1, 2], [2, 4]]$
- $A_{\text{yeni}} = [[1,0],[0,1]] + [[1,2],[2,4]] = [[2,2],[2,5]]$
- $b_{\text{yeni}} = [0,0] + 1 \times [1,2] = [1,2]$

Eğer reward 0 gelseydi, A yine aynı şekilde güncellenecekti (çünkü context yine 'görüldü') ama b değişmeyecekti: $b_{\text{yeni}} = [0,0] + 0 \times [1,2] = [0,0]$.

5.4 Bu güncellemeler sonraki kullanıcıları nasıl etkiler? Algoritma nerede öğreniyor?

Bir sonraki kullanıcı geldiğinde döngü baştan çalışır, ama artık $A[\text{arm}]$ ve $b[\text{arm}]$ eskisi gibi değildir — güncellenmiş halleriyle theta yeniden hesaplanır. Eğer bir arm sürekli olumlu sonuç veren context'lerle karşılaşmışsa, b o yönde büyür ve theta bu yönü daha güçlü şekilde yansıtır; böylece benzer context'ler için prediction yükselir. Aynı zamanda A büyüdükçe (arm daha çok denendikçe) o context yönünde uncertainty

küçülür. Yani 'öğrenme' tek bir fonksiyonda değil, tam olarak bu iki satırın her iterasyonda A ve b'yi biriktirmesinde gerçekleşir — sonraki her tahmin, önceki tüm gözlemlerin toplamı üzerinden yapılır.

6. Sonuçların Hesaplanması

```
selected = np.array(chosen_arms) == arm
total_selected = np.sum(selected)
total_reward = np.sum(np.array(obtained_rewards)[selected])
reward_rate = total_reward / total_selected
```

chosen_arms, 8.000 kullanıcı boyunca hangi arm'ın seçildiğini sırayla tutan bir listedir (örn. [2, 0, 2, 4, ...]). selected == arm karşılaştırması, bu listeyi True/False değerlerinden oluşan bir 'maske'ye çevirir: ilgili arm'ın seçildiği yerlerde True, diğerlerinde False. np.sum(selected) True değerlerini sayar (Python'da True = 1 olarak toplanır) ve o arm'ın toplam kaç kez seçildiğini verir. np.array(obtained_rewards)[selected] ifadesi boolean indexing'tir: obtained_rewards listesindeki değerlerden yalnızca selected'ta True olan konumlar seçilir; bunların toplamı, o arm'ın topladığı toplam reward'ı (kaç kez 1 aldığını) verir. reward_rate ise bu ikisinin oranıdır — arm seçildiğinde ne sıklıkla başarılı (reward=1) olduğunu gösterir.

Bu veri seti ve kod ile üretilen gerçek sonuçlar:

Arm	Seçilme Sayısı	Toplam Reward	Başarı Oranı
0	2.283	1.011	%44,3
1	993	273	%27,5
2	3.136	1.311	%41,8
3	1.014	301	%29,7
4	574	101	%17,6

Görüldüğü gibi algoritma zamanla arm_2 ve arm_0'ı çok daha sık seçmiş (toplam 5.419 kez, tüm seçimlerin %68'i), çünkü bu iki arm'ın başarı oranı diğerlerinden belirgin şekilde yüksek. Bu, LinUCB'nin exploration ile başlayıp giderek daha çok exploitation'a kaymasının somut bir göstergesidir: baştaki A = birim matris sayesinde tüm arm'lar başlangıçta eşit şansla denenmiş, ama veri biriktikçe düşük performanslı arm'lar (özellikle arm_4, %17,6) giderek daha az seçilmiştir.

7. Python / NumPy Notları (Bu Projedeki Kullanımlarıyla)

Kullanım	Bu koddaki anlamı
append()	chosen_arms.append(chosen_arm) gibi kullanımlarda, listenin sonuna yeni bir eleman ekler. Örn. her kullanıcıdan sonra o kullanıcının seçilen arm'ı ve aldığı reward listeye eklenir.
range()	for i in range(len(X)) ifadesinde 0'dan 7999'a kadar sayı üretir; böylece X'teki her kullanıcı sırayla i indeksiyle işlenir. for arm in range(n_arms) ise 0-4 arası 5 arm'ı dolaşır.

Kullanım	Bu koddaki anlamı
.shape	Bir numpy dizisinin boyutlarını verir. $X.shape \rightarrow (8000, 15)$: 8000 kullanıcı, 15 feature. $x.shape \rightarrow (15,)$: tek kullanıcının vektör uzunluğu.
np.array()	Python listesini (örn. chosen_arms, obtained_rewards) numpy dizisine çevirir; bu sayede == arm gibi eleman-eleman karşılaştırma ve toplu matematiksel işlemler yapılabilir hale gelir.
np.dot()	İki vektörün iç çarpımını (skaler sonuç) ya da matris-vektör çarpımını yapar. $np.dot(x, \theta)$ $\rightarrow$ prediction (tek sayı). $np.dot(A_{inv}, x)$ $\rightarrow$ matris-vektör çarpımı (vektör sonuç).
np.outer(x, x)	İki vektörden dış çarpım ile bir matris üretir; burada x'in kendisiyle dış çarpımı, o context'in A matrisine eklenecek 'bilgi katkısını' oluşturur.
np.sqrt()	Karekök alır; uncertainty_value her zaman negatif olmadığından, standart sapmaya benzer bir belirsizlik ölçüsüne çevirmek için kullanılır.
np.linalg.inv()	Bir matrisin tersini (A^{-1}) doğrudan hesaplar; burada $x^T A^{-1} x$ ifadesindeki A^{-1} 'i elde etmek için kullanılır.
np.linalg.solve()	$A\theta = b$ denklem sistemini, A'nın tersini açıkça hesaplamadan, daha kararlı biçimde çözer ve θ 'yı döndürür.
Indexing / boolean indexing	$X[i]$ ile i. kullanıcının satırı alınır (tekil indexing). $np.array(obtained_rewards)[selected]$ ile yalnızca selected maskesinde True olan elemanlar seçilir (boolean indexing) — bu, bir if/for döngüsü yazmadan filtreleme yapmanın numpy yoludur.

En Önemli Noktalar

- LinUCB, her arm için ayrı bir A matrisi ve b vektörü tutarak, reward'ın context'e doğrusal bağlı olduğunu varsayan bir öğrenme yapısıdır.
- A, 'bu arm hakkında ne kadar veri biriktirdik' bilgisini; b ise 'bu veriler ne yönde sonuç verdi' bilgisini taşır.
- $\theta = np.linalg.solve(A, b)$, $A\theta = b$ denklemini çözerek o arm için öğrenilen katsayıları verir.
- $prediction = x \cdot \theta$, mevcut bilgiyle context için beklenen reward'ı tahmin eder.
- $uncertainty = \sqrt{x^T A^{-1} x}$, bu tahminin ne kadar az veriye dayandığını, yani ne kadar güvenilmez olabileceğini ölçer.
- $score = prediction + \alpha \cdot uncertainty$, exploitation (bilineni kullan) ile exploration (bilinmeyeni dene) arasında matematiksel bir denge kurar.
- Her adımda en yüksek score'a sahip arm seçilir (argmax) ve kullanıcıya gösterilir.
- Gerçek reward gözlemlendikten sonra yalnızca seçilen arm'ın A'sına $np.outer(x, x)$, b'sine $reward \cdot x$ eklenir — öğrenme tam olarak burada gerçekleşir.
- Bu güncellemeler, bir sonraki benzer kullanıcı geldiğinde θ 'yı (ve dolayısıyla prediction ile uncertainty'yi) değiştirir; algoritma bu şekilde deneyim biriktirir.

- Genel akış: context → prediction → uncertainty → score → arm seçimi → reward → A/b update → yeni kullanıcı — bu döngü veri setindeki 8.000 kullanıcı boyunca tekrar eder ve zamanla daha başarılı arm'lara doğru kayar (arm_0 ve arm_2'nin toplam seçimlerin %68'ini alması bunun kanıtıdır).

Kod Değerlendirmesi

Kodun kendisi LinUCB algoritmasını doğru ve okunabilir şekilde uyguluyor; A/b güncellemeleri, solve/inv kullanımı ve sonuç hesaplama mantığı algoritmayla tam örtüşüyor. Çalışan mantığı değiştirmeden, ileride dikkat edilebilecek birkaç teknik nokta:

- StandardScaler, online döngü başlamadan önce tüm 8.000 satır üzerinde fit ediliyor (scaler.fit_transform(df_processed[numeric_cols])). Bu, ileride gelecek kullanıcıların istatistiklerinin (ortalama, standart sapma) baştan bilinmesi anlamına gelir — hafif bir 'geleceği görme' (data leakage) durumu. Gerçek bir online sistemde scaler'ın da zamanla (veya sadece geçmiş veriyle) güncellenmesi daha doğru olur; bu proje bir simülasyon/öğrenme amaçlı olduğu için bu bir hata değil, ileri seviye bir iyileştirme notudur.
- Her arm için uncertainty hesaplanırken np.linalg.inv(A[arm]) ile tam matris tersi açıkça alınıyor. theta için solve() kullanılırken, uncertainty için de A_inv'i hiç oluşturmadan np.linalg.solve(A[arm], x) ile ara sonucu bulup x ile iç çarpımını almak, büyük feature sayılarında hem daha hızlı hem sayısal olarak daha kararlı olurdu. 15 feature için bu fark ihmal edilebilir düzeydedir, kodu değiştirmeye gerek yoktur.
- alpha = 1 sabit tutulmuş; exploration/exploitation dengesi hiç ayarlanmamış. Farklı alpha değerleriyle (örn. 0.5 veya 2) denenip toplam reward'ın nasıl değiştiği gözlemlenebilir; bu, algoritmanın davranışını daha iyi anlamak için iyi bir sonraki adım olur.